LEXICON

# Algorithm & Flowchart

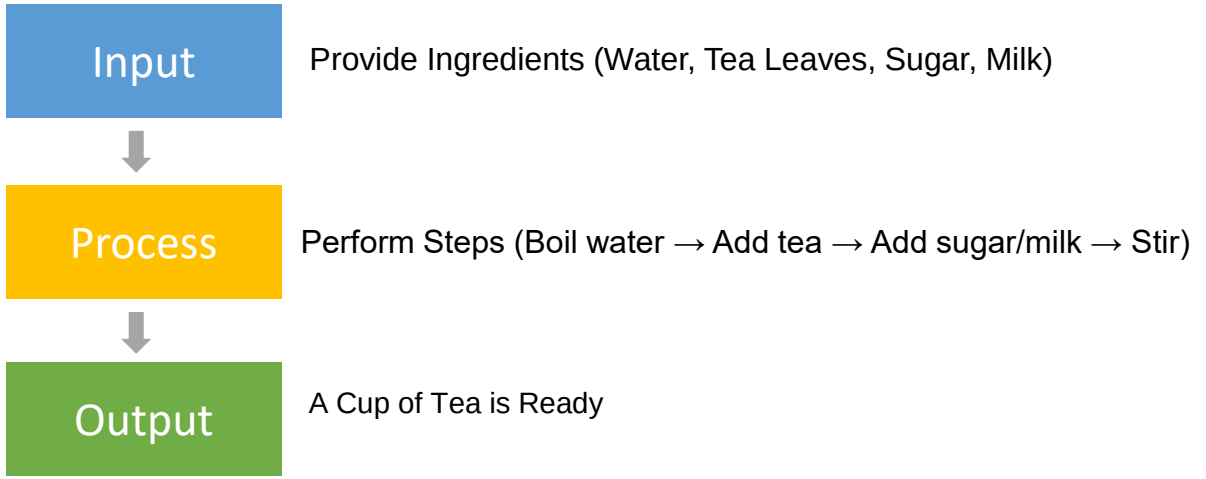
Foundations of Problem Solving: Algorithm, Pseudocode, and Flowchart

# Objectives

- Understand what an **algorithm** is and why it is important in problem-solving.
- Explain what a **pseudocode** is and write simple examples.
- Describe what a **flowchart** is and identify common symbols.
- **Convert** a problem from algorithm → pseudocode → flowchart.
- Apply logical thinking to **design solutions** before writing code.
- Recognize how these skills form the **foundation for programming** in languages like Java.

# What is an Algorithm?

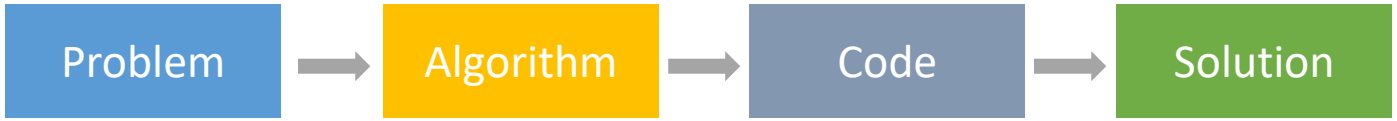
- An **algorithm** is a step-by-step process **(sequence of instructions)** to solve a problem
- Describes **what needs to be done** — not how the computer will do it.
- Are the **foundation of all programming** — every program you write follows some algorithm.
- Use algorithms in our **daily routines**:
  - Getting ready in the morning
  - Cooking a meal
  - Choosing a route to school or work
- Algorithms often include **decisions**




*If you can explain the steps, you're already writing an algorithm.*

# Why Algorithm Matter in Problem-Solving

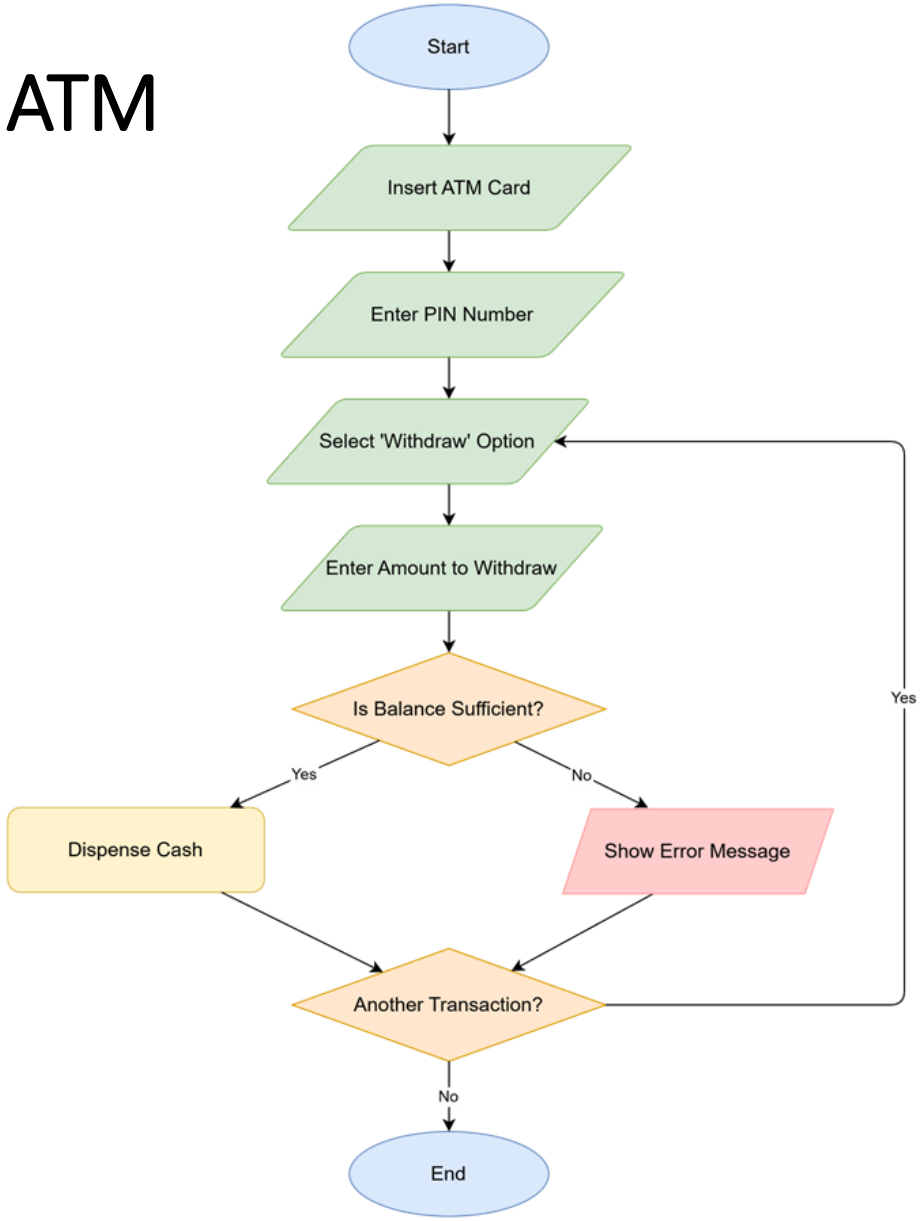
- **Organize your thinking:** Break down big problems into smaller, clear steps.
- **Ensure accuracy:** Reduces mistakes by planning logic before coding.
- **Improve efficiency:** Helps you design solutions that are faster and smarter.
- **Language-independent:** Can be applied in any programming language (like Java, Python, C++).
- **Essential skill:** Forms the base for pseudocode, flowcharts, and actual code.




*A well-designed algorithm is half the solution — coding is just translating it.*

# Steps to Withdraw Money from an ATM

1.Start
2.Insert ATM card
3.Enter PIN number
4.Select “Withdraw” option
5.Enter the amount to withdraw
6.If the balance is sufficient → dispense cash
7.If the balance is insufficient → show error message
8.Ask if user wants another transaction
9.If yes → go to Step 4
10.If no → eject card and end



# What is Pseudocode

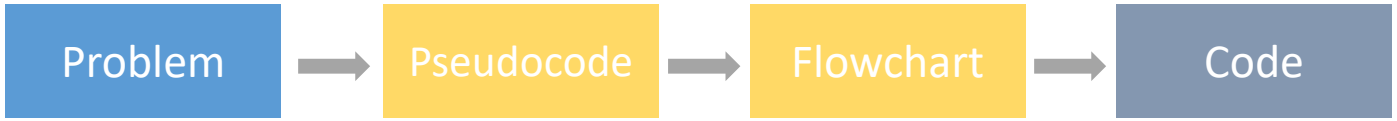
- **Simple, Human-Readable Way** to describe the steps of an algorithm.
- Uses **plain text statements,** not exact programming syntax.
- Tool to **think logically** and **plan solution** before coding.
- Makes it easier to **translate** an algorithm into a programming language like Java.

```
Algorithm: Make a Cup of Coffee
1. Start
2. Boil water
3. Add coffee powder to a cup
4. Pour hot water into the cup
5. Add sugar and milk (if needed)
6. Stir well
7. Serve the coffee
8. End
```

*Pseudocode is the bridge between your idea and actual code*

# Why Use Pseudocode?

- **Clarifies thinking:** Helps you focus on *logic*, not syntax.
- **Easy to write and read:** Anyone can understand the steps.
- **Language-independent:** Can be turned into Java, Python, C++, etc.
- **Reduces coding errors:** You solve problems *before* writing code.
- **Excellent planning tool:** Makes algorithm-to-code conversion easy.



# Guidelines for Writing Pseudocode

- Use **simple words** - **sentences** and clear steps.
- Write **one step per line**.
- Use keywords like: *Start, Input, If, Then, Else, While, Display, End.*
- Indent loops and conditions for readability.
- Avoid programming syntax — focus on logic.

```
1. Start
2. Input number N
3. If N % 2 == 0 Then
       Display "Even"
   Else
       Display "Odd"
4. End
```

# Pseudocode and Problem-Solving Flow

- Step 1: Understand the problem
- Step 2: Break it into steps (algorithm)
- Step 3: Write pseudocode for those steps
- Step 4: Convert pseudocode into a flowchart (optional)
- Step 5: Translate pseudocode into code (Java, C#, Python, etc.)

*Pseudocode is a great thinking tool — but not a programming language.*

# Practice – Writing Pseudocode

Write a **step-by-step pseudocode** using plain text and logical structure.

### **Exercise 1: Find the Largest of Two Numbers** *(Decision)*

Write the **pseudocode** for a program that:

- ❑ Takes **two numbers** A and B as input.
- ❑ Compares the two numbers.
- ❑ Displays which number is **larger**.
- ❑ If they are equal, display **"Both numbers are equal."**

### **Exercise 2: Sum of 5 Numbers** ***(****Loop + Accumulation****)***

Write pseudocode for a program that:

- ❑ Reads **5 numbers** one by one.
- ❑ Calculates their **total sum**.
- ❑ Displays the result.

# Transition to Flowcharts

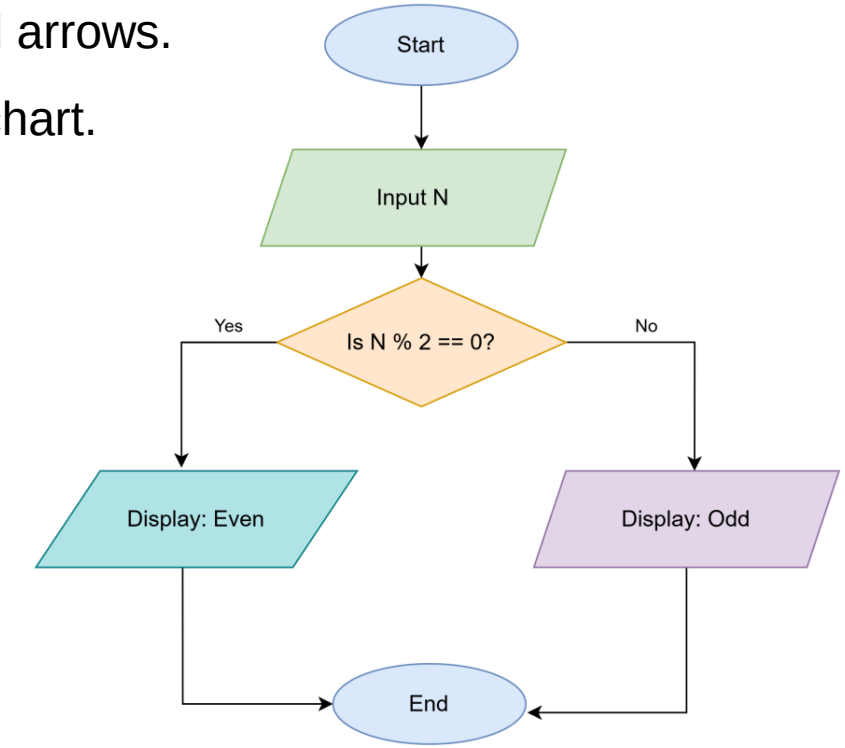
Pseudocode describes the **logic in words**.

A flowchart shows the **same logic visually** using shapes and arrows.

Each pseudocode step usually matches a symbol in the flowchart.

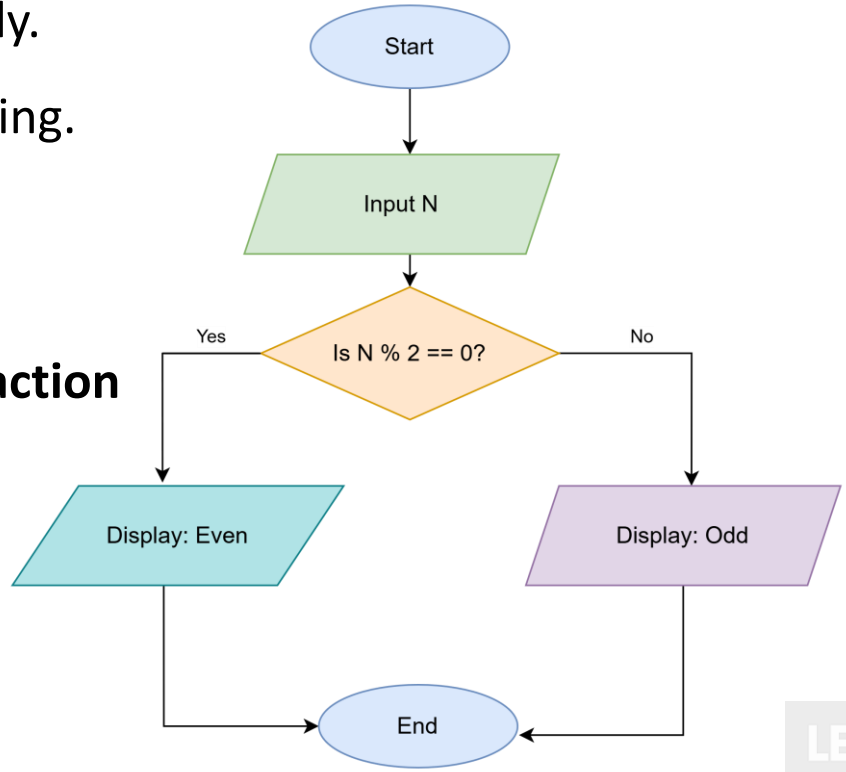
Flowcharts make it easier to understand and debug the logic.

```
1. Start
2. Input number N
3. If N % 2 == 0 Then
       Display "Even"
   Else
       Display "Odd"
4. End
```

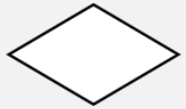


# What is a Flowchart?

- **A visual diagram** that shows the **steps of an algorithm** using symbols and arrows.
- Represents the **flow of control**, how the process moves from one step to another.
- Helps to **understand, design, and explain** a process clearly.
- Make it easy to **spot errors** and **improve logic** before coding.
- Simplifies complex logic - Easy to read and communicate
- Useful for debugging and documentation
- Each symbol in a flowchart represents a **specific type of action** (start, input/output, process, or decision).



# Common flowchart symbols

| Symbol | Name | Usage / Description |
|---|---|---|
| **Oval**  | **Start / End** | Marks the **beginning or end** of a process or algorithm. |
| **Diamond**  | **Decision** | Used for **Yes/No** or **True/False** decisions that branch the flow. |
| **Parallelogram**  | **Input / Output** | Represents **data entry (input)** or **display of results (output)**. |
| **Rectangle**  | **Process / Operation** | Indicates a **processing step** or **action**, such as calculations. |
| **Connector**  | **Connector** | Used to **link flow lines** when the diagram continues elsewhere. |
| **Predefined Process**  | **Subprocess** | Refers to a **predefined group of steps** (a reusable process). |
| **Arrow**  | **Flow of Control** | Shows the **direction** or **sequence** of steps in the process. |

# How to draw and read a flowchart

- Start with a **clear algorithm or pseudocode**.
- Identify the **inputs**, **processes**, **decisions**, and **outputs**.
- Use **appropriate symbols** for each type of step.
- Connect the symbols with **arrows** to show the logical flow.
- Always **start** and **end** with the terminal (oval) symbol.
- Keep the flowchart **simple and easy to follow** (top to bottom).

# Practice – Designing Flowchart

**1. A program asks the user to enter their age.**

- If the age is **18 or older**, display the message **“You are eligible to vote.”**
- If the age is **less than 18**, display the message **“You are not eligible to vote.”**
- Then **end the program.**

**2. A program takes a student’s marks (out of 100) as input.**

- If marks are **90 or above**, display **“Grade A”**
- If marks are **between 75 and 89**, display **“Grade B”**
- If marks are **between 50 and 74**, display **“Grade C”**
- If marks are **below 50**, display **“Fail”**
- End the program.

*Use the correct symbols and show the logical flow clearly (Start → Process → Decision → End)*

# Practice – Designing Flowchart

**3. A program that calculates the sum of 5 numbers entered by the user.**

- The program should **ask the user** to enter 5 numbers one by one.
- It should **keep a running total** of the numbers.
- After all numbers are entered, the program should **display the total sum**.
- Then, **end the program**.

**4. A program takes three numbers (A, B, and C) as input.**

- Determine which number is the **largest**.
- Display the **largest number**.
- End the program.

*Use the correct symbols and show the logical flow clearly (Start → Process → Decision → End)*

# Why Algorithm and Flowchart?

- Builds Logical Thinking
- Simplifies Programming
- Avoids Syntax Confusion
- Helps in Debugging and Understanding
- Foundation for Any Programming Language

*If you can design the logic clearly, coding becomes just translation.*

# Types of Flowcharts

| Type of Flowchart | Purpose / Description |
| --- | --- |
| **Program Flowchart** | Shows the **logic and sequence of operations** inside a specific program or algorithm. |
| **System Flowchart** | Describes how **data moves through a system** — includes inputs, processes, storage, and outputs. |
| **Process Flowchart** | Represents a **business or daily workflow** — steps in an operation or routine. |
| **Document Flowchart** | Tracks the **flow of documents or information** within an organization. |
| **Data Flow Diagram (DFD)** | Focuses on **data movement** between processes, stores, and external entities. |